

FlowMemory Uniswap v4 Hooks

A Public Technical Breakdown of a Memory-Native DeFi Primitive

FlowMemory

May 19, 2026

- FlowMemory Uniswap v4 Hooks
 - Public Technical Report
 - Abstract
 - One-Page Launch Summary
 - What Was Created
 - Architecture
 - Evidence And Performance
 - Why This Is Different
 - Why It Is Better On Its Chosen Axis
 - Scientific Contribution
 - What This Does Not Claim
 - Reproducibility
 - Publication-Ready Summary
 - References

FlowMemory Uniswap v4 Hooks

Public Technical Report

Core line

```
The swap is not the memory.  
The transaction is the proof envelope.  
The FlowPulse is the memory artifact.
```

FlowMemory introduces a memory-native Uniswap v4 hook primitive: a verified on-chain emission boundary where DeFi execution can produce FlowPulse memory signals.

This report explains what was created, how it works, how it performed under local verification, why it is different from ordinary Uniswap v4 hook demos, how it extends into AI-agent and GPU-workflow memory, and what claims remain explicitly pending.

The strongest technical thesis is:

FlowMemory is not trying to make agents remember more.
It is making impossible memory states fail.

Abstract

Most DeFi infrastructure is built around execution: swaps, settlement, liquidity, fees, routing, and accounting. Most AI memory infrastructure is built around retrieval: context search, vector databases, summaries, and persistent state. FlowMemory connects a different axis: **receipt-bound memory consistency**.

This repository implements and documents a narrow Uniswap v4 `afterSwap` hook that emits a `FlowPulse` memory signal after a completed swap boundary. The hook does not custody funds, route swaps, control fees, change accounting, or claim receipt metadata during execution. It emits the memory artifact. The transaction receipt becomes the proof envelope. Reader and verifier infrastructure attach receipt-derived facts such as `txHash` and `logIndex` after the transaction lands.

The repository then expands that boundary into a public R&D model for machine memory. FMM-o, the FlowMemory Agent Memory Model, checks whether a machine history could have happened around receipt-bound `FlowPulse` boundaries. `FlowLitmus` and the FMM-o harnesses execute forbidden-outcome tests: pre-receipt receipt reads, retrocausal claims, stale output survival, rootfield rollback, split-brain writes, and unsafe cache or compute reuse.

The result is not another hook demo. It is a first public step toward memory-native DeFi and proof-backed machine memory: execution boundaries become memory artifacts, and downstream systems can reject histories that should be impossible.

One-Page Launch Summary

Area	Launch-prep status
Repository	Public GitHub repository: <code>FlowmemoryAI/flowmemory-uniswap-v4-hooks</code>
Verification snapshot	036d0c2 before report packaging; the report commit is recorded in Git history
Public prerelease	v0.1.0-launch-prep
License	MIT
Latest CI at verification snapshot	Passing
Solidity tests	12 passed
Python/tool tests	276 passed
Launch reality check	PASS
Public claim gate	PASS: 0 unguarded overclaims
Local FMM-0 consistency surface	PASS
Public Base Sepolia receipt evidence	PENDING
Production verifier infrastructure	NOT_CLAIMED

The project has a deliberately narrow public claim surface:

1. `afterSwap` is the verified on-chain emission boundary.
2. `FlowPulse` is the memory artifact.
3. The transaction is the proof envelope.
4. FMM-0 is the memory consistency model.
5. Cache and compute reuse are blocked unless lineage and FMM-0 consistency pass.
6. Public Base Sepolia receipt evidence remains pending until a real release packet validates.

What Was Created

1. Memory-Native Uniswap v4 Hook

Primary contracts:

- `contracts/FlowMemoryAfterSwapHook.sol`
- `contracts/FlowPulse.sol`

- `contracts/FlowMemoryHookPlanner.sol`
- `contracts/interfaces/IFlowMemoryHookData.sol`
- `contracts/interfaces/IUniswapV4SwapHookLike.sol`

The hook is intentionally small. Its job is not to make a swap smarter. Its job is to make the swap boundary memorable.

Contract invariants:

- `afterSwap` is the intended callback.
- The callback is gated to the configured `PoolManager`.
- `hookData` is required.
- `rootfieldId` is required.
- `commitment` is required.
- `sender` is required.
- The hook returns the correct `afterSwap` selector.
- The hook returns zero hook delta.
- No custody path exists.
- No dynamic fee path exists.
- No routing path exists.
- No custom accounting path exists.
- `FlowPulse` does not include `txHash`, `transactionIndex`, or `logIndex`.
- Receipt metadata is reader-derived after execution.
- URI content is advisory and untrusted.
- The actor may be a router or contract sender, not necessarily an end-user EOA.

2. FlowPulse Memory Artifact

`FlowPulse` is the memory signal emitted from the hook boundary.

For swap memory, the pulse includes:

- `pulseId`
- `rootfieldId`
- `actor`
- `pulseType`
- `subject`

- `commitment`
- `parentPulseId`
- `sequence`
- `occurredAt`
- `uri`

It intentionally excludes receipt-only metadata. The hook cannot know a transaction hash or log index during EVM execution. Those fields are attached later by a reader from the transaction receipt.

3. FMM-0: FlowMemory Agent Memory Model

FMM-O reframes AI-agent memory as a consistency model rather than a retrieval store.

It asks:

Could this machine history have happened?

FMM-O introduces local launch machinery for:

- phase-state classification;
- forbidden transition checks;
- adversarial counterexample generation;
- memory algebra closure;
- boundary bisimulation;
- forbidden-core extraction;
- witness packaging.

4. FlowLitmus

FlowLitmus is the executable forbidden-outcomes suite.

It runs adversarial machine histories against the FlowMemory runtime model:

Case	Meaning	Expected
FM-LB-001	Pre-receipt txHash read	fault
FM-SER-001	Retrocausal receipt claim	fault
FM-QS-001	Unquiesced post-boundary output	fault
FM-RT-001	Speculative output escaped	fault
FM-FIS-001	Stale output survived boundary	fault
FM-SER-002	Rootfield rollback	fault
FM-SER-003	Split-brain canonical write	fault
FM-OK-001	Valid boundary history	pass

All eight cases pass in the launch-prep suite.

5. AI/GPU Workflow Memory Gates

The repository also adds three executable R&D artifacts for AI and GPU workflow reuse:

- Cache Lineage Gate
- Compute Reuse Router
- Compute Reuse Consistency

These do not claim GPU hardware acceleration. They test a safer systems property:

FlowMemory is not making the chip faster.
It is making compute reuse harder to get wrong.

The gates check whether cache and compute reuse are allowed only when lineage, fingerprints, freshness, attestation policy, and receipt-bound memory history are compatible.

6. Public Claim Gate

Because this project uses bold category language, it also includes a launch-copy guard:

```
python tools/public_claim_gate.py --pretty
```

Current result:

```
files checked: 8
guarded risk mentions: 68
unguarded overclaims: 0
Public launch copy is claim-safe.
```

This gate exists to enforce explicit do-not-claim boundaries. It fails if public copy asserts live Base mainnet deployment, audited custody, fund protection, GPU acceleration, semantic truth, model correctness, or hook-time receipt metadata outside explicit non-claim contexts.

Architecture

FlowMemory separates four layers:

```
Execution Layer
  Uniswap v4 PoolManager
  swap lifecycle
  afterSwap boundary

Memory Emission Layer
  FlowMemoryAfterSwapHook
  FlowPulse
  rootfieldId
  commitment
  parentPulseId
  URI

Evidence Layer
  EVM logs
  transaction receipt
  txHash
  transactionIndex
  logIndex
  finality policy
  reader-derived metadata

Memory Layer
  FlowMemory reader
  verifier policy
  FMM-0
  Rootflow downstream memory state
```

The core path:

```
swap executes
-> PoolManager calls afterSwap
-> FlowMemoryAfterSwapHook validates the boundary
-> hook emits AfterSwapObserved
-> hook emits FlowPulse
-> transaction receipt lands
-> reader attaches txHash/logIndex/finality
-> FMM-0 checks whether downstream machine history is possible
```

The hook emits the memory signal. The reader proves where it landed.

Evidence And Performance

This repository does not claim live production performance. It reports local verification performance: deterministic tests, proof gates, and CI status.

Verification Results

At verification snapshot 036d0c2, before report packaging, the launch-prep verification surface was:

Verification command	Result
<code>python -m unittest discover -s tools -p 'test_*.py'</code>	276 tests passed
<code>forge fmt --check</code>	passed
<code>forge build</code>	passed
<code>forge test -vvv</code>	12 tests passed
<code>python tools/flowmemory_release_transcript.py -p</code> <code>--pretty</code>	local surface PASS; public evidence PENDING
<code>python tools/public_claim_gate.py --pretty</code>	0 unguarded overclaims
<code>python tools/launch_reality_check.py --pretty</code>	launch reality PASS
<code>python tools/compute_reuse_consistency.py demo</code> <code>--pretty</code>	5/5 cases passed; 4/4 unsafe reuse blocked
GitHub Actions CI	passing

Release Transcript Result

FMM-0 Witness Pack	PASS	8/8 local layers, 0 escaped faults
Launch Reality Check	PASS	boundary model, hook invariants, artifacts, and FlowLitmus
Compute Reuse Router	PASS	1 reuse, 4/4 unsafe reuse rejected
Cache Lineage Gate	PASS	1 cache reuse, 4/4 unsafe reuse rejected
Compute Reuse Consistency	PASS	5/5 cases, 4/4 unsafe reuse blocked
Public Base Sepolia Receipt Evidence	PENDING	

Compute Reuse Consistency Result

```

CRC-OK-001  PASS  ACCEPT_REUSE
CRC-KV-001  PASS  BLOCK_CACHE_REUSE
CRC-GPU-001 PASS  RUN_GPU_JOB
CRC-SER-001 PASS  BLOCK_IMPOSSIBLE_HISTORY
CRC-ATT-001 PASS  BLOCK_CACHE_REUSE

```

```

cases passed: 5/5
unsafe reuse blocked: 4/4

```

The important metric is not throughput. The important metric is that unsafe reuse is rejected deterministically when lineage, runtime, attestation policy, or receipt-bound history drifts.

Why This Is Different

Compared With Ordinary Uniswap v4 Hooks

Uniswap v4 hooks can execute custom logic at lifecycle points such as before or after a swap. Most public hook categories focus on execution modification:

- fee logic;
- liquidity behavior;
- routing;
- incentives;
- custom accounting;
- MEV behavior;
- trading mechanics.

FlowMemory is different.

It does not try to make the swap cheaper, faster, more routed, or more financially expressive. It makes the completed execution boundary emit memory.

Most hooks modify execution.
FlowMemory emits memory.

That makes the hook small by design. The narrowness is not lack of ambition. It is the security boundary that makes the primitive credible.

Compared With Blockchain Indexers And Dashboards

Indexers and dashboards usually observe logs after the fact. They make history searchable or visible.

FlowMemory does something different: it emits an intentional memory artifact at the protocol boundary and then lets downstream systems verify where it landed.

The memory signal is not invented later by analytics. It is emitted at the execution boundary through explicit hook data.

Compared With RAG And Vector Memory

Retrieval-augmented generation combines model memory with non-parametric retrieval over external stores. That is useful, but the core question is usually what context should be retrieved.

FlowMemory asks another question:

Could this memory history have happened?

That question matters for autonomous systems. Agents are no longer just chat interfaces. They run tools, call models, manage caches, perform transactions, reuse compute, and write state across external events. The dangerous failure is not only hallucination. It is impossible history.

FlowMemory is better on this axis because it gives machine memory an external boundary and a consistency test. It does not merely retrieve relevant context. It blocks histories that violate receipt-bound reality.

Compared With GPU Memory Optimizations

Modern AI infrastructure already has strong work on GPU memory and cache reuse:

- TensorRT-LLM supports KV-cache reuse for shared prefixes.
- vLLM and PagedAttention target KV-cache waste and sharing.

- FlashAttention reduces GPU HBM/SRAM IO for attention.
- NVIDIA attestation can speak to hardware and runtime trust boundaries.

FlowMemory does not replace any of that.

It sits around it:

GPU memory handles tensors.
FlowMemory handles lineage, commitments, proof envelopes, and reuse discipline.

The GPU angle is workflow-level:

- Do not rerun compute if a compatible committed artifact already exists.
- Do not reuse a cache if tokenizer, runtime, adapter, side input, policy, or lineage drifted.
- Do not trust compute reuse if the receipt-bound machine history is impossible.

That is not hardware acceleration. It is proof-backed compute discipline.

Why It Is Better On Its Chosen Axis

No system is better at everything. FlowMemory is better on a specific and important axis:

It makes memory verifiable against public execution boundaries.

That gives it four advantages over ordinary memory infrastructure.

1. Boundary-Native Memory

Most systems store observations. FlowMemory creates a protocol-level memory signal at the boundary where execution completes.

2. Receipt-Separated Evidence

The hook does not pretend to know receipt facts. It emits a signal. The receipt later proves where the signal landed.

This separation is stronger than systems that blur local guesses, event logs, and final receipt facts into one memory layer.

3. Executable Forbidden Outcomes

FlowMemory does not only describe what should happen. It runs local adversarial cases and requires impossible histories to fail.

That moves the project from narrative to conformance.

4. Proof-Backed Reuse Discipline

AI and GPU systems increasingly depend on reuse: retrieved context, KV caches, prior model outputs, prior embeddings, prior compute, and long-running agent state.

Reuse is only safe when lineage is compatible. FlowMemory turns that rule into deterministic gates.

Scientific Contribution

This launch-prep package can be read as a public research artifact around one claim:

Memory for autonomous systems should have consistency rules, not only retrieval.

The project contributes:

1. A concrete DeFi boundary where a memory signal is emitted.
2. A receipt-separated proof envelope model.
3. A memory artifact schema that excludes impossible hook-time receipt fields.
4. A draft memory consistency model for machine histories.
5. Executable forbidden-outcome tests.
6. A proof-backed cache and compute reuse discipline.
7. Public non-claim gates that keep the launch language technically bounded.

This is why the project is more than marketing. A reviewer can run the commands and see which histories pass, which histories fail, and which claims remain pending.

What This Does Not Claim

The project deliberately does **not** claim:

- live Base mainnet deployment;
- audited custody infrastructure;
- fund protection;

- swap control;
- fee control;
- routing control;
- custom accounting;
- semantic truth;
- model correctness;
- GPU hardware speedup;
- production verifier readiness;
- hook-time knowledge of txHash, transactionIndex, or logIndex.

Public Base Sepolia receipt evidence is still **PENDING**. A live release requires a separate release record with deployment transaction, source verification, observed logs, reader evidence, and receipt-derived metadata.

Reproducibility

Use this command sequence first:

```
python tools/flowmemory_release_transcript.py --pretty
python tools/public_claim_gate.py --pretty
python tools/launch_reality_check.py --pretty
python tools/compute_reuse_consistency.py demo --pretty
forge test -vvv
```

Expected public status:

```
Local FMM-0 consistency surface: PASS
Public Base Sepolia receipt evidence: PENDING
Production verifier infrastructure: NOT_CLAIMED
```

The printable PDF is generated from this Markdown source with Pandoc and the committed stylesheet:

```
pandoc FLOWMEMORY_PUBLIC_TECHNICAL_REPORT.md --standalone --embed-resources --toc --toc-depth=2 --
metadata lang=en --css docs/PUBLIC_TECHNICAL_REPORT.css -o
cache/FLOWMEMORY_PUBLIC_TECHNICAL_REPORT.html
```

Publication-Ready Summary

FlowMemory introduces a memory-native Uniswap v4 hook primitive. The swap is not the memory; the transaction is the proof envelope; the FlowPulse is the memory artifact. The hook is deliberately narrow: `afterSwap`, PoolManager gated, zero hook delta, no custody, no routing, no fee engine, no custom accounting, and no receipt metadata smuggling.

The broader contribution is FMM-o, a receipt-bound memory consistency model for machine histories. Instead of treating agent memory as retrieval alone, FlowMemory checks whether a claimed machine history could have happened around public execution boundaries. FlowLitmus and the FMM-o harnesses make that claim executable by catching forbidden outcomes.

For AI and GPU workflows, the same pattern becomes proof-backed reuse discipline. FlowMemory does not make GPUs faster. It makes unsafe cache and compute reuse fail when lineage, runtime, attestation policy, or receipt-bound history drift.

The category is simple:

```
DeFi has execution.  
AI has retrieval.  
FlowMemory adds memory consistency.
```

References

1. Uniswap Developers, “Deployments.” Used for current Uniswap v4 deployment addresses and the Base Sepolia PoolManager assumption:
<https://developers.uniswap.org/contracts/v4/deployments>
2. Uniswap Developers, “Hook Deployment.” Used for address-encoded hook permission and address-mining behavior:
<https://developers.uniswap.org/docs/protocols/v4/guides/hooks/hook-deployment>
3. Uniswap Developers, “Hooks.” Used for the `beforeSwap` and `afterSwap` lifecycle framing: <https://developers.uniswap.org/contracts/v4/reference/core/libraries/Hooks>
4. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” Used to frame retrieval-oriented AI memory: <https://arxiv.org/abs/2005.11401>
5. NVIDIA TensorRT-LLM, “KV cache reuse.” Used to frame existing KV-cache reuse and shared-prefix reuse behavior: <https://nvidia.github.io/TensorRT-LLM/advanced/kv-cache-reuse.html>
6. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention.” Used to frame KV-cache paging and sharing:

<https://arxiv.org/abs/2309.06180>

7. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.” Used to distinguish GPU IO optimization from FlowMemory’s proof-backed reuse discipline: <https://arxiv.org/abs/2205.14135>
8. NVIDIA Attestation SDK, “Attestation.” Used to distinguish hardware/runtime attestation from FlowMemory’s memory-artifact layer:
https://docs.nvidia.com/attestation/attestation-client-tools-sdk/latest/gpu_and_switch_attestation.html